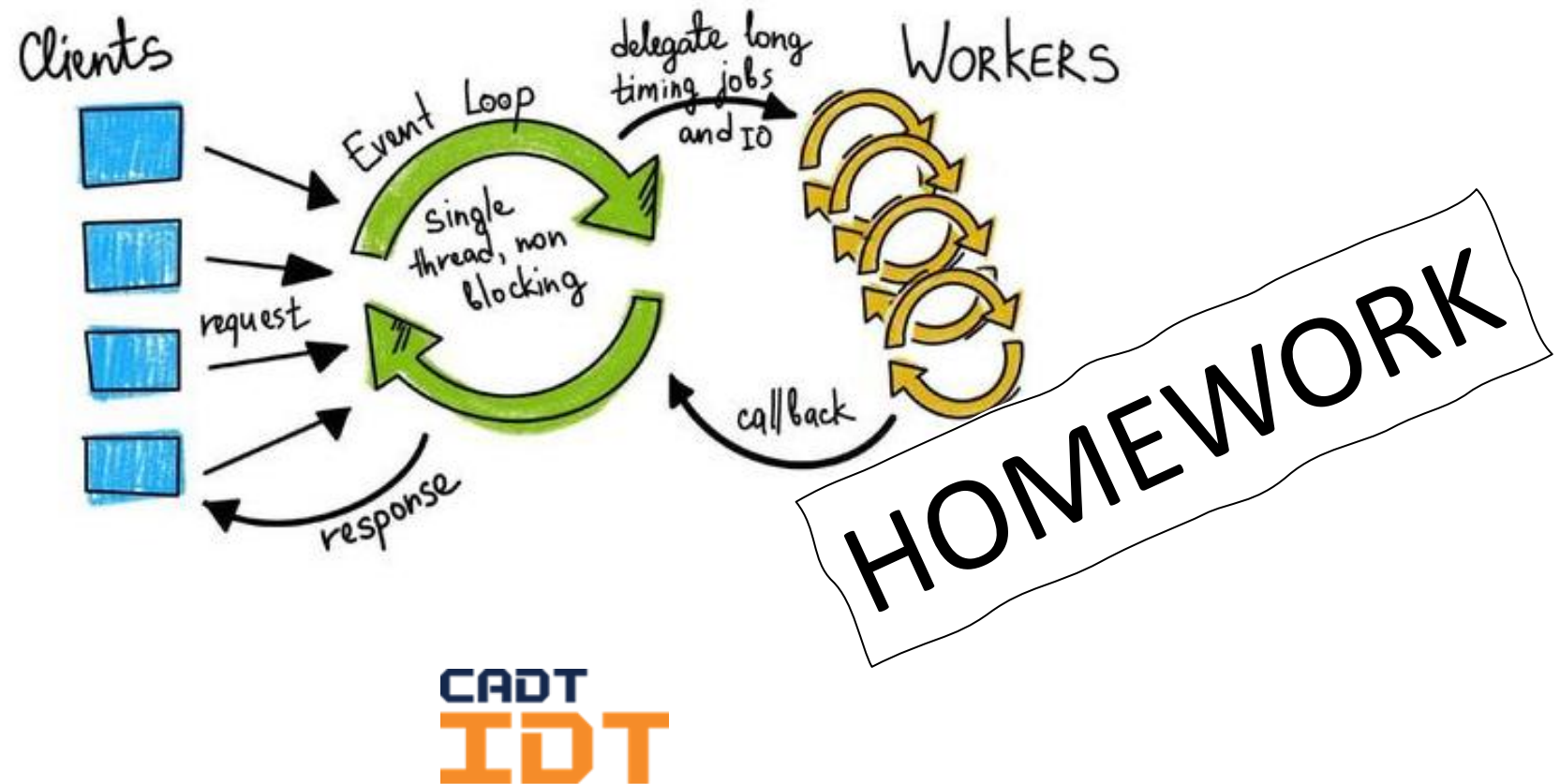
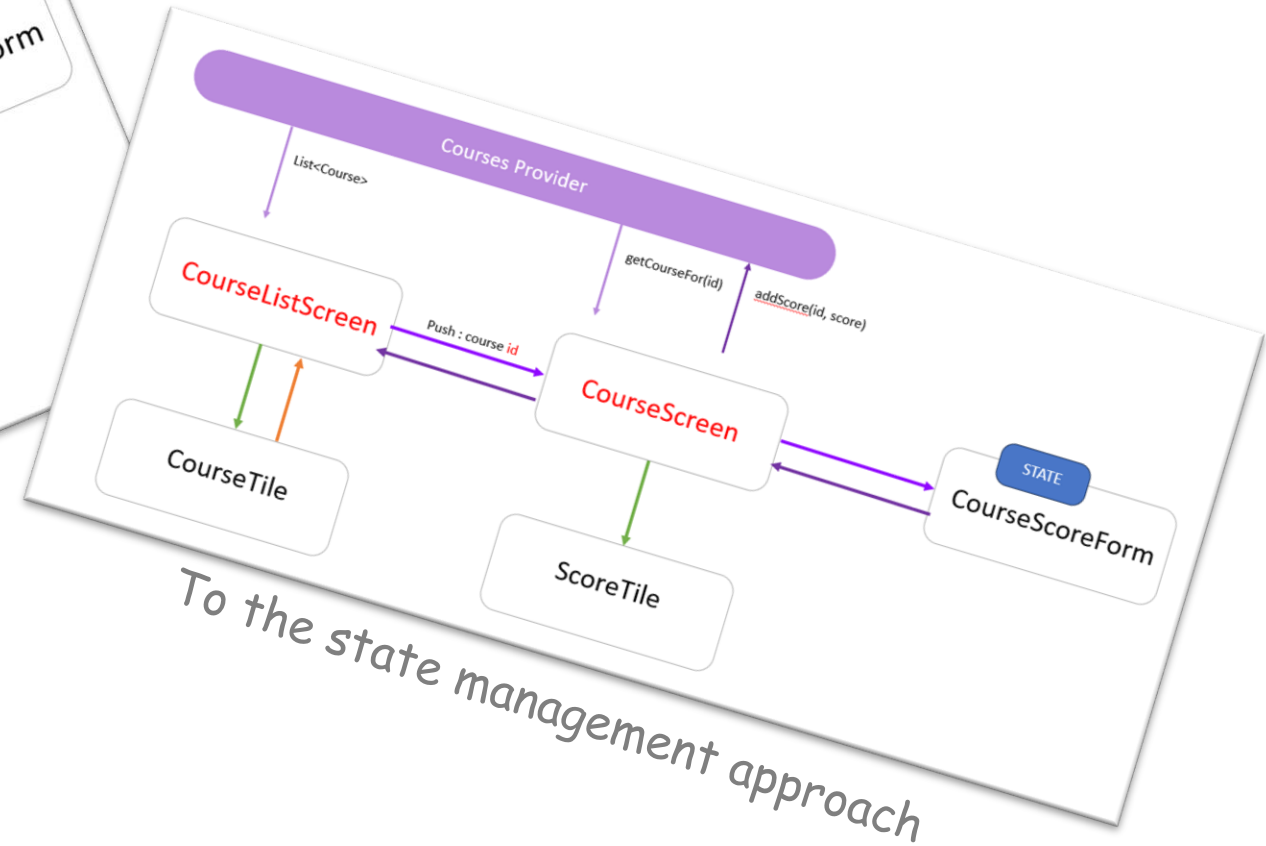
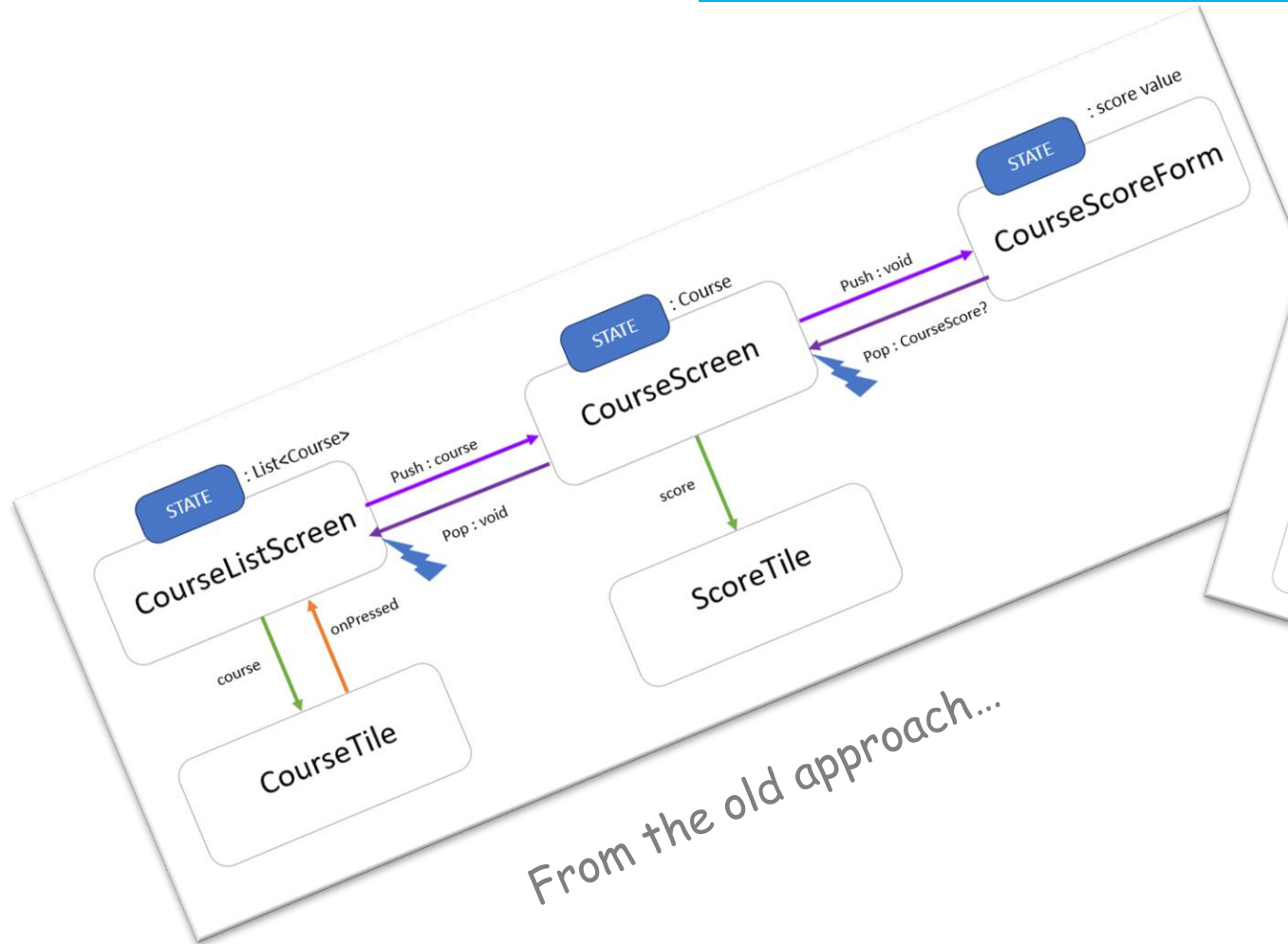


MOBILE DEVELOPMENT 2

W6 – Asynchronous Calls & Networking

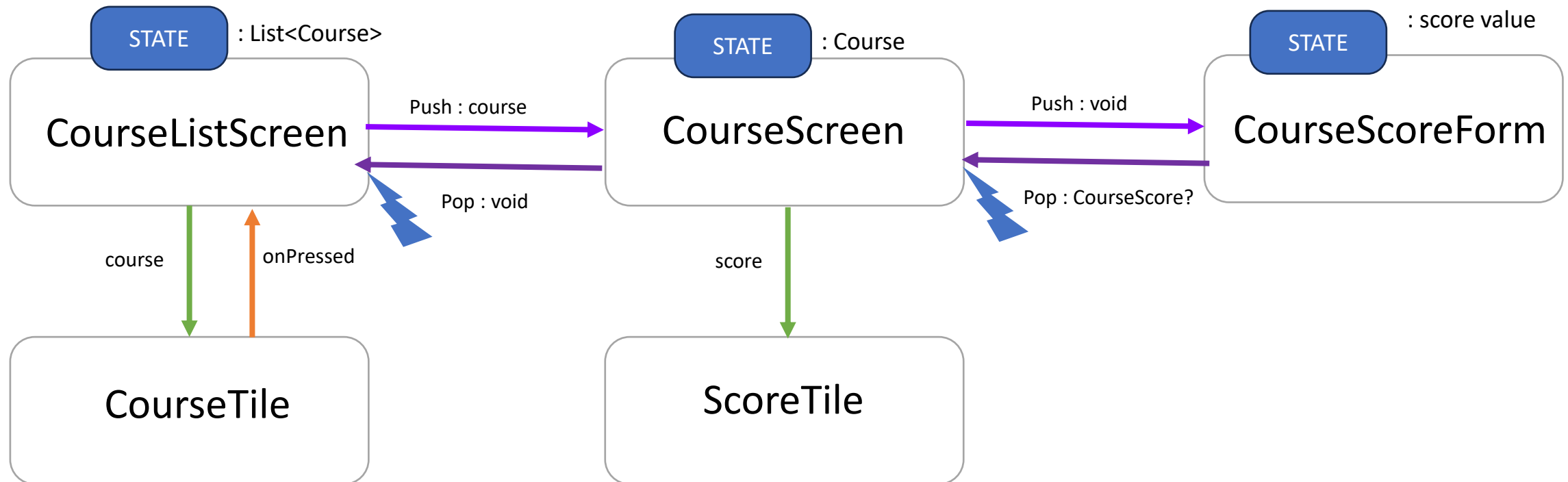


EXERCICE 1



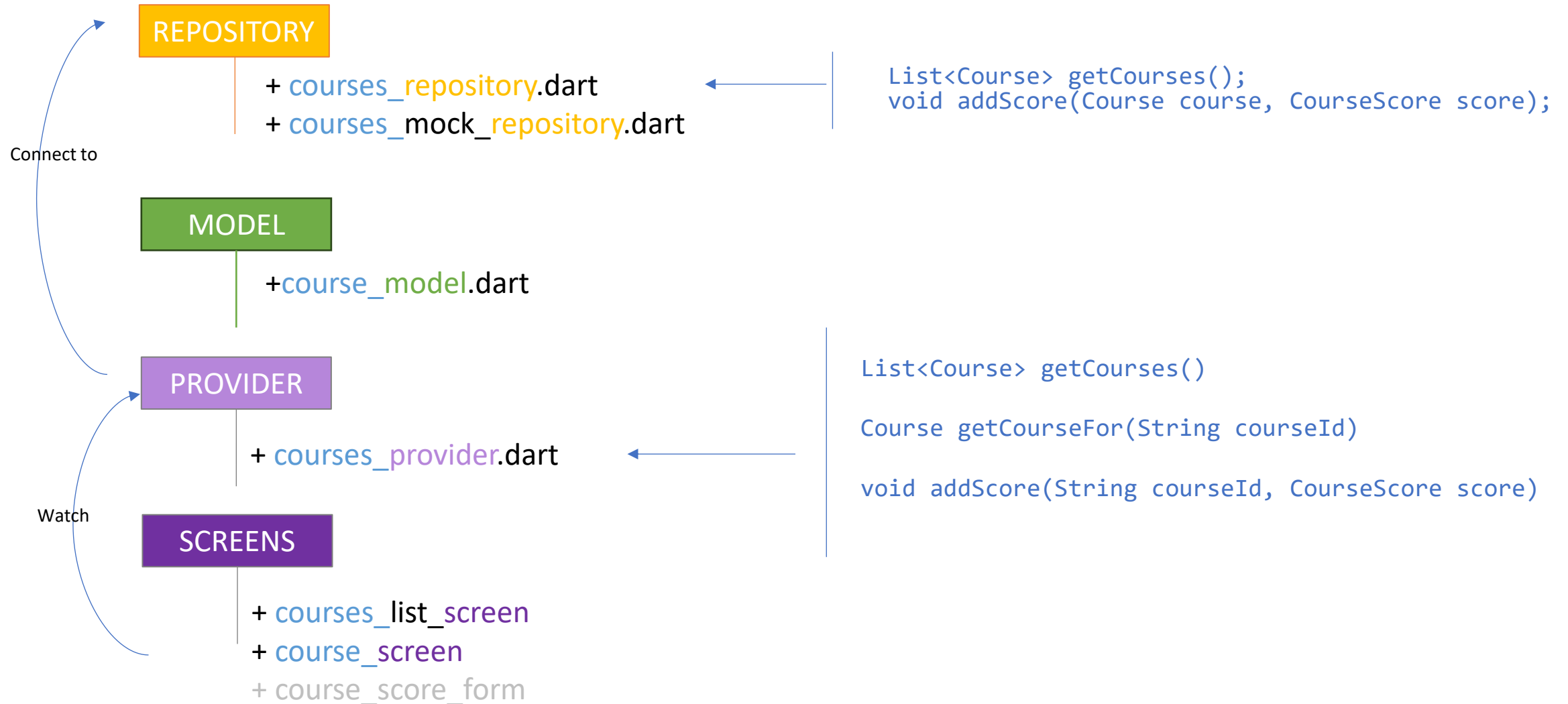
Start Code

You start code is the correction of the Term 1 EXAM



Expected Code Structure

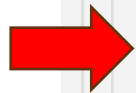
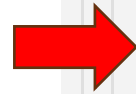
You need to structure the new code as described bellow



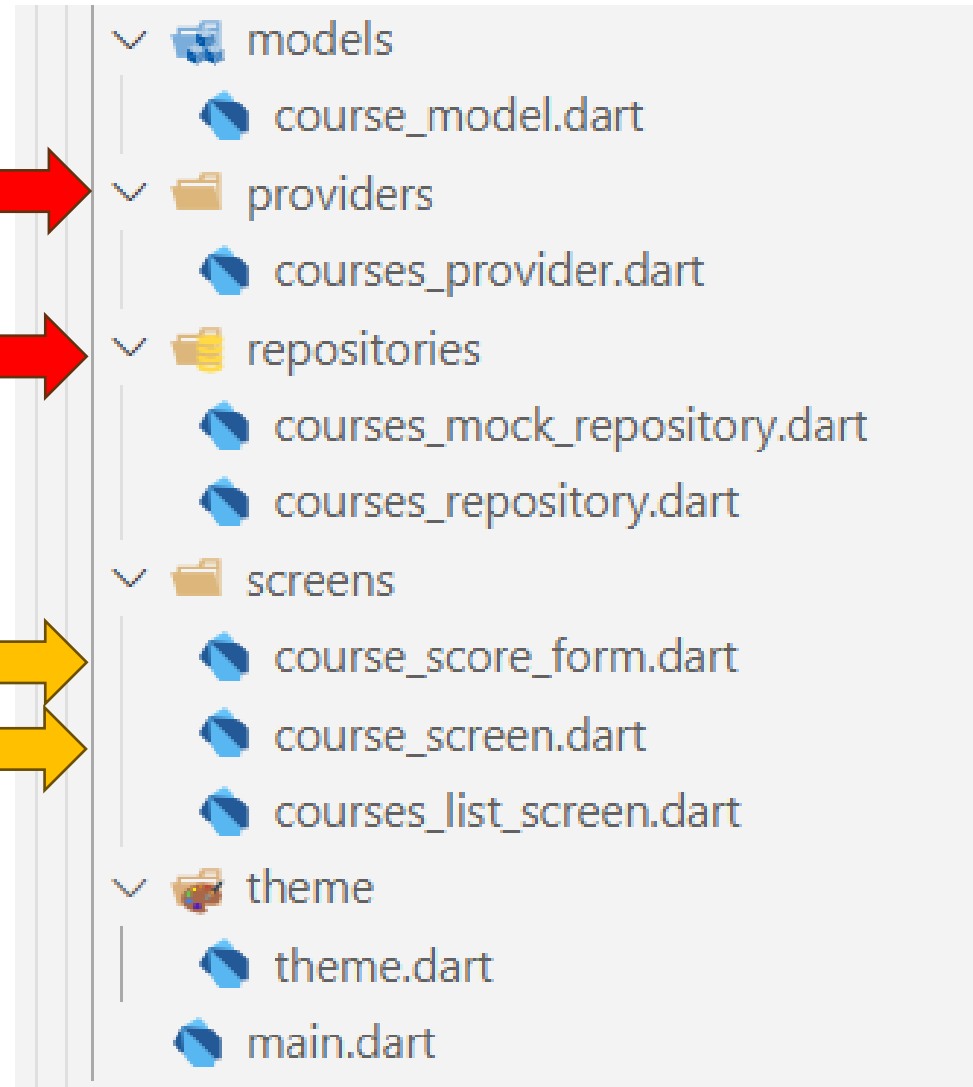
Expected Code Structure

You need to structure the new code as described bellow

TO CREATE !

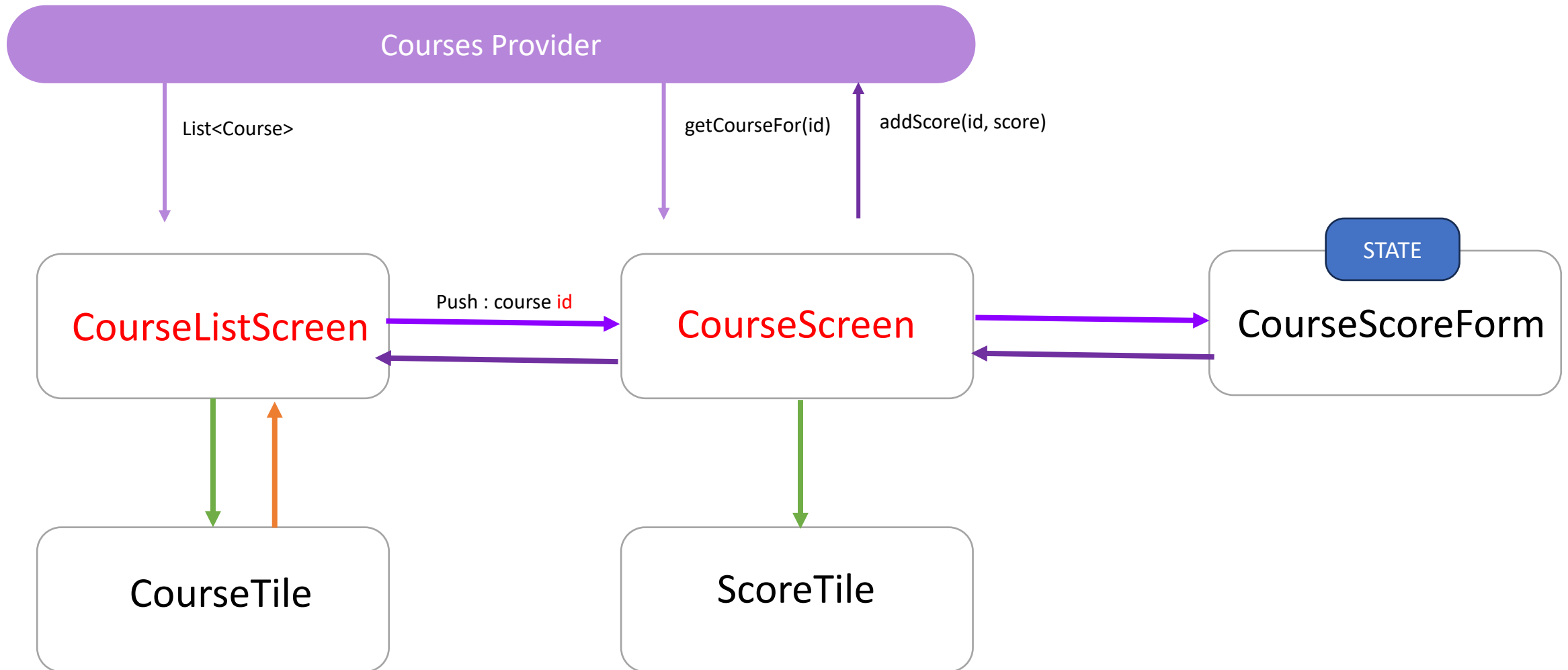


TO UPDATE !



Expected Components Interaction

You need to implement the provider and interact with it, as described bellow



EXERCICE 2

PART 1

Manage a **List of Posts**

Our current code displays only 1 post ! Update the code to fetch a list of posts

1. Update the abstract and mock repository

- Update the `getPosts()` to return a `Future<List<Post>>`
- For the mock we return a fake list after 5 seconds

2. Update the provider

- Now you get a list of post as value of the Future, store it to the Async Data...

3. Update the screen

- Now we need to display the list of post instead of a single post
- We still have the 3 cases : no async data / error / loading / success
- You can add another case : the list is returned, but empty ! We can display "No"

REPOSITORY

+ `posts_repository.dart`
+ `mock_posts_repository.dart`

MODEL

+ `post.dart`

PROVIDER

+ `posts_provider.dart`

SCREEN

+ `posts_screen.dart`

PART 2

Implement the **HTTP** repository

Let's connect to JSON place holder API

1. Implement the HTTP posts repository

- ✓ This repository shall get the posts from given REST API
<https://jsonplaceholder.typicode.com/posts>

2. Implement a `fromJson` method to create a Post

- ✓ In the `post_http_repository`, create a method
`Post.fromJson(Map<String, dynamic> json)`
- ✓ This method should return a new `Post` instance, given the JSON properties
- It should throw an exception if any parsing issue (ex missing field...)
- Note: you can use such checks: `assert(json['id'] is int);`

3. Change the `main()` repo to http repo

- 4. Test the UI still works well with real data
- ✓ Test different scenario (ex: no internet...)

DATA

DTO

+ `post_dto.dart`

REPOSITORY

+ `posts_repository.dart`
+ `http_posts_repository.dart`

MODEL

MODEL

+ `post.dart`

UI

PROVIDER

+ `posts_provider.dart`

SCREEN

+ `posts_screen.dart`

Manage a List of Posts

Our current code displays only 1 post ! Update the code to fetch a list of posts

REPOSITORY

- + posts_repository.dart
- + **mock_posts_repository.dart**

MODEL

- + post.dart

PROVIDER

- + **posts_provider.dart**

SCREEN

- + **posts_screen.dart**

1. Update the abstract and mock repository

- Update the getPosts() to return a `Future<List<Post>>`
- *For the mock we return a fake list after 5 seconds*

2. Update the provider

- Now you get a list of post as value of the Future, store it to the Async Data...

3. Update the screen

- Now we need to display the list of post instead of a single post
 - We still have the 3 cases : no async data / error / loading / success
 - You can add another case : the list is returned, but empty ! We can display “No posts for now”

Let's connect to **REST APIs**

The Flutter http package is a Dart library for making HTTP requests, handling REST APIs, and communicating with web services.



pub.dev

<https://pub.dev/packages/provider>

```
$ flutter pub add http
```

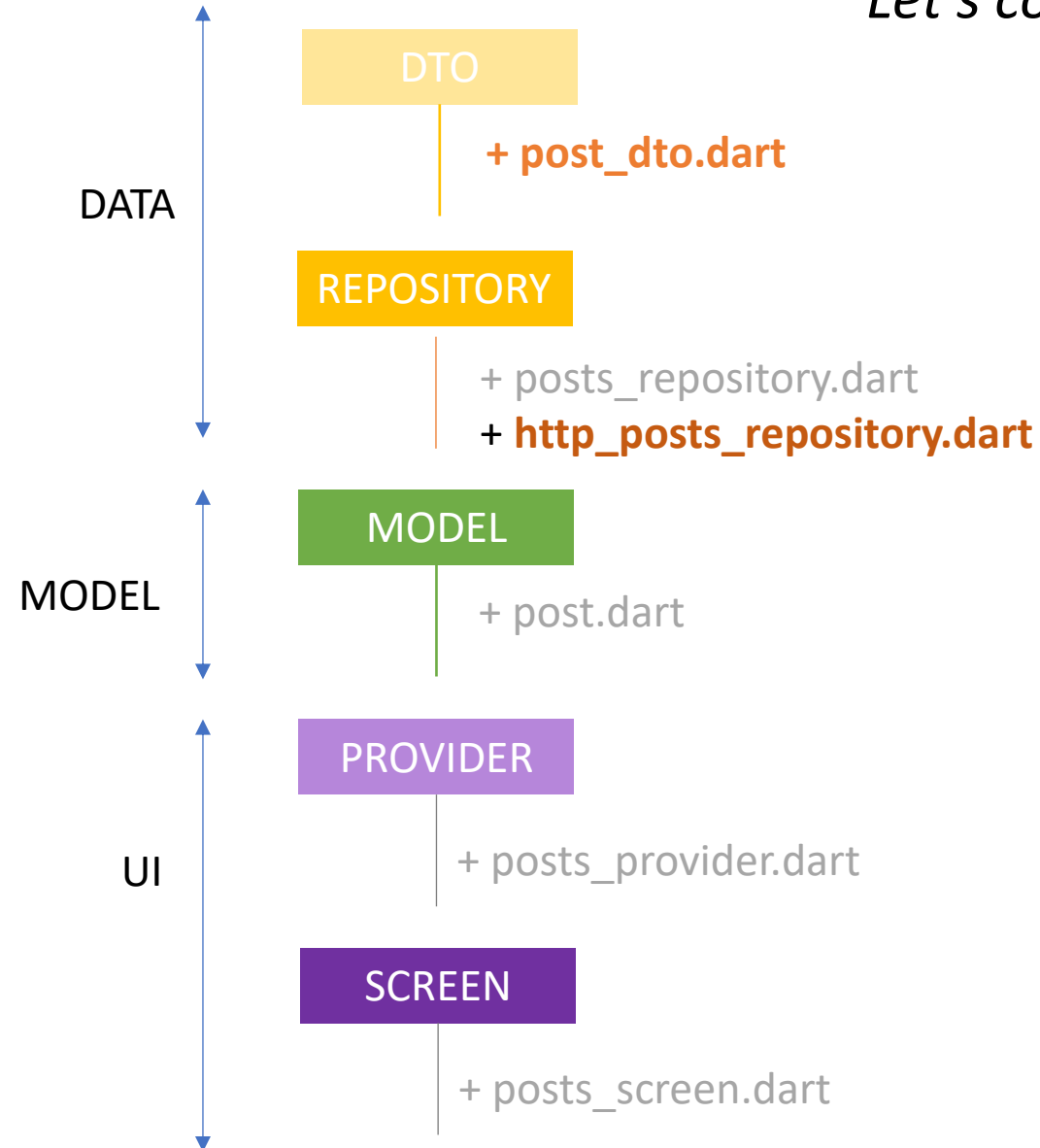
To include the package provider in your project

```
$ flutter pub get
```

To update the dependencies in your computer

Implement the **HTTP** repository

Let's connect to JSON place holder API



1. Implement the HTTP posts repository

- ✓ This repository shall get the posts from given REST API <https://jsonplaceholder.typicode.com/posts>

2. Implement a fromJson method to create a Post

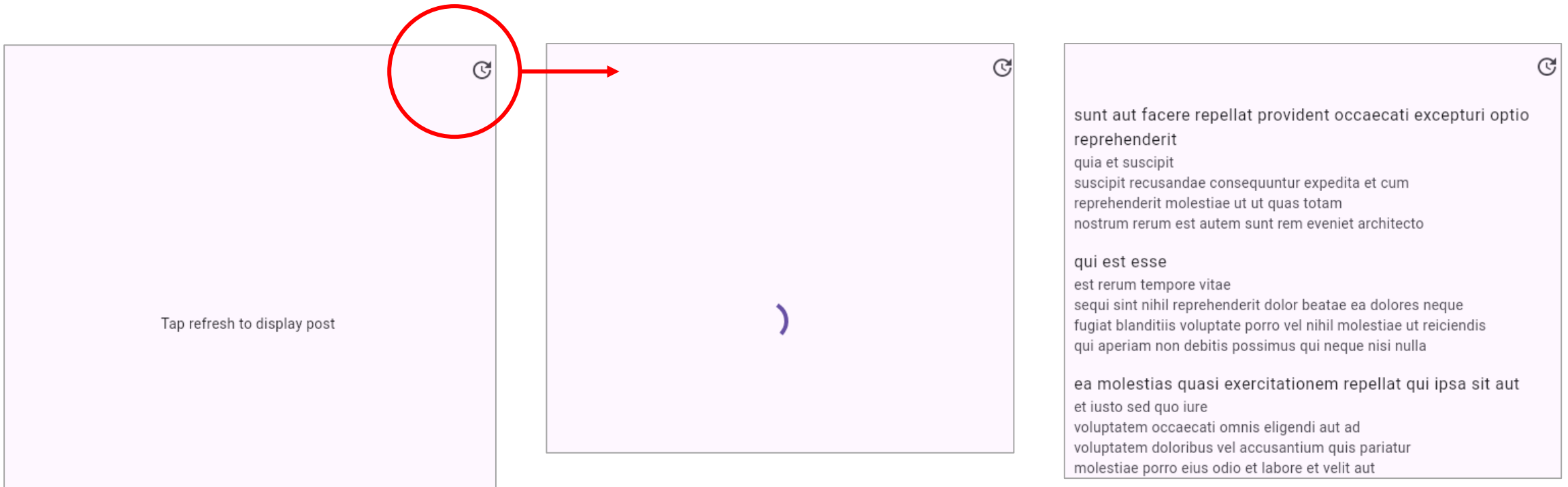
- ✓ In the `post_http_repository`, create a method `Post fromJson(Map<String, dynamic> json)`
- ✓ This method should return a new `Post` instance, given the JSON properties
 - It should throw an exception if any parsing issue (ex missing field...)
 - Note: you can use such checks: `assert(json['id'] is int);`

3. Change the main() repo to http repo

4. Test the UI still works well with real data

- ✓ Test different scenario (ex: no internet..)

Expected Result



Posts are retrieved from [the API](#)